Person B – Step-by-step guide (Data & NoSQL)

MediSentinel · Phase 1 · Your folder: **ingestion/** · Lead: Rishi (Person A) reviews every pull request

Part 0 – One-time setup (about 45 minutes)

0.1 Install these first

Software	Where
Git	git-scm.com (keep the default options)
Python 3.13	python.org – tick "Add python.exe to PATH"
PostgreSQL 18	postgresql.org/download/windows – remember the **postgres** password you choose
MongoDB 8 Community + Compass	mongodb.com/try/download/community – install as a Windows service
VS Code + Claude Code	code.visualstudio.com, then the Claude Code extension (or the Claude desktop app)

0.2 Accept the invite and clone

Open the GitHub invite e-mail from Rishi and click **Accept**. Then:

Run in the terminal:

```
git clone https://github.com/Rishi799/MediSentinel.git
```

Run in the terminal:

```
cd MediSentinel
```

Open the **MediSentinel** folder in VS Code and start Claude Code in it.

0.3 Let Claude set up your machine

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo:
https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:

- Only change files inside my own folder (ingestion/) and tests/. Never edit sql/,
medisentinel/ or other people's folders.

- Never commit .env, passwords, .venv or downloaded data.

- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit
messages or pull requests.

- Use parameterized SQL only (%s placeholders), never f-strings with values.
```

```
- Explain what you did in simple words after each step so I can explain it in the viva.


Task: set up this repo on my Windows machine by following README.md ("Setup" section) step by
step.

1. Create the .venv with Python 3.13, install requirements.txt and run "pip install -e .".

2. Create .env from .env.example. Ask me to choose three passwords for MS_DOCTOR_PASSWORD,

   MS_PHARMACIST_PASSWORD and MS_ADMIN_PASSWORD (do not invent them yourself).

3. Build the database: create the medisentinel database, run sql/schema.sql,
sql/seed_demo.sql and

   sql/roles.sql (with the same three passwords). When the postgres password is needed, ask
me to type it.

4. Run "python tools/check_setup.py" and "python -m pytest".

If anything fails, find the reason and fix it, then run both checks again until everything is
OK.
```

Done when: check_setup prints *All good* and pytest shows all tests passed. If you are stuck, send the error to Rishi.

0.4 Read your task sheet

Paste this prompt into Claude Code:

```
Read docs/tasks/PERSON_B.md, CONTRIBUTING.md and docs/PHASE1_SPRINT.md.

Explain my tasks to me in simple words: what each task produces, in which order I do them,

and which shared functions from medisentinel/ I will use. Do not write any code yet.
```

Task B1 – openFDA data into MongoDB

Branch: `feature/openfda-ingestion` · **When:** Day 1 morning

Step 1 – start the branch

Run in the terminal:

```
git checkout main

git pull

git checkout -b feature/openfda-ingestion
```

Step 2 – let Claude build it

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo:
https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:

- Only change files inside my own folder (ingestion/) and tests/. Never edit sql/,
medisentinel/ or other people's folders.

- Never commit .env, passwords, .venv or downloaded data.
```

```
- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit messages or pull requests.

- Use parameterized SQL only (%s placeholders), never f-strings with values.

- Explain what you did in simple words after each step so I can explain it in the viva.


Do Task B1 from docs/tasks/PERSON_B.md: create ingestion/load_openfda.py.

- Download drug labels that have an interaction section from

  https://api.fda.gov/drug/label.json?search=_exists_:drug_interactions (limit 1000 per request, use skip for more)

- Download adverse-event reports from https://api.fda.gov/drug/event.json

- Store raw JSON in MongoDB (use medisentinel.config.MONGO_URI and MONGO_DB): labels in collection

  medication_labels with _id = label "id"; reports in adverse_reports with _id = "safetyreportid".

  Use bulk_write with ReplaceOne(..., upsert=True) so re-running never duplicates.

- Create indexes: medication_labels on openfda.generic_name and openfda.rxcui;

  adverse_reports on patient.drug.medicinalproduct and patient.reaction.reactionmeddrapt.

- Command-line options --labels (default 1000) and --reports (default 5000); wait ~0.3 s between

  requests, retry on HTTP errors, print progress and final counts.

Then run it with the defaults and show me the counts.
```

Step 3 – check it yourself

Open **MongoDB Compass** → connect to `mongodb://localhost:27017` → database **medisentinel**. You should see about 1000 documents in **medication_labels** and 5000 in **adverse_reports**, and the indexes in the *Indexes* tab. Run the script a second time – the counts must not double.

Step 4 – submit

Paste this prompt into Claude Code:

```
Run "python -m pytest" and make sure everything passes. Then show me "git status" and commit only my

task's files with a clear message (no Co-Authored-By line, no mention of Claude). Push the branch

feature/openfda-ingestion to origin and give me the GitHub link to open the pull request.
```

Open the link → **Create pull request** → tell Rishi. Wait for him to merge before starting the next task (or start the next branch from main if he says so).

Step 5 – understand it for the viva

Paste this prompt into Claude Code:

```
Explain ingestion/load_openfda.py to me like a viva examiner would ask: why the openFDA data is stored in
```

```
MongoDB instead of PostgreSQL, what an upsert is and why we use it, what the indexes are for, and what

limit/skip do in the API. Then ask me 3 viva questions and check my answers.
```

Task B2 – Fill the SQL Medication table from MongoDB

Branch: `feature/medication-loader` · **When:** Day 1 afternoon

Step 1 – start the branch

Run in the terminal:

```
git checkout main

git pull

git checkout -b feature/medication-loader
```

Step 2 – let Claude build it

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo: https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:

- Only change files inside my own folder (ingestion/) and tests/. Never edit sql/, medisentinel/ or other people's folders.

- Never commit .env, passwords, .venv or downloaded data.

- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit messages or pull requests.

- Use parameterized SQL only (%s placeholders), never f-strings with values.

- Explain what you did in simple words after each step so I can explain it in the viva.


Do Task B2 from docs/tasks/PERSON_B.md: create ingestion/load_medications.py.

- Write a pure function label_to_medication(label) -> dict | None (no database inside it):

  name = openfda.brand_name[0] (fallback generic_name[0]) in Title Case, max 150 chars;

  generic_name = openfda.generic_name[0] lower case, max 150; rxcui = openfda.rxcui[0], max 20;

  drug_class = openfda.pharm_class_epc[0] without the trailing " [EPC]", max 100 (may be None).

  Return None if there is no generic name.

- Read all labels from MongoDB, convert them, and insert into PostgreSQL using

  medisentinel.db.connect_as("ms_admin") with INSERT ... ON CONFLICT (rxcui) DO NOTHING.

  Skip a label whose generic_name already exists in Medication (case-insensitive) so the demo medicines

  like Warfarin are not duplicated.

- Print labels read, inserted, skipped.
```

```
- Add tests/test_ingestion.py testing label_to_medication: normal label, missing brand name,
  missing generic name -> None, "[EPC]" removed, long name truncated.
Run the script and the tests and show me the results.
```

Step 3 – check it yourself

In pgAdmin (or ask Claude to run it) check: `SELECT count(*) FROM medication;` should now be several hundred, and `SELECT name FROM medication WHERE generic_name ILIKE 'warfarin%';` must not show duplicates.

Step 4 – submit

Paste this prompt into Claude Code:

```
Run "python -m pytest" and make sure everything passes. Then show me "git status" and commit
only my
task's files with a clear message (no Co-Authored-By line, no mention of Claude). Push the
branch
feature/medication-loader to origin and give me the GitHub link to open the pull request.
```

Open the link → **Create pull request** → tell Rishi. Wait for him to merge before starting the next task (or start the next branch from main if he says so).

Step 5 – understand it for the viva

Paste this prompt into Claude Code:

```
Explain ingestion/load_medications.py for the viva: how it moves data from NoSQL (MongoDB)
into SQL
(PostgreSQL), why label_to_medication is a separate pure function, what ON CONFLICT DO
NOTHING does and why
the RxCUI is UNIQUE. Then ask me 3 viva questions and check my answers.
```

Task B3 + B4 – Synthetic test data and MongoDB demo queries

Branch: `feature/synthetic-data` · **When:** Day 2 morning

Step 1 – start the branch

Run in the terminal:

```
git checkout main
git pull
git checkout -b feature/synthetic-data
```

Step 2 – let Claude build it

Paste this prompt into Claude Code:

```
You are helping me on the MediSentinel DBMS lab project (repo:
https://github.com/Rishi799/MediSentinel).

Rules for everything you do in this session:

- Only change files inside my own folder (ingestion/) and tests/. Never edit sql/,
medisentinel/ or other people's folders.

- Never commit .env, passwords, .venv or downloaded data.

- When you commit, do NOT add any "Co-Authored-By" line or any mention of Claude/AI in commit
messages or pull requests.

- Use parameterized SQL only (%s placeholders), never f-strings with values.

- Explain what you did in simple words after each step so I can explain it in the viva.


Do Tasks B3 and B4 from docs/tasks/PERSON_B.md.

B3 – ingestion/generate_synthetic.py with options --patients 100 --prescriptions 300 --seed
42:

- Faker("en_IN") seeded with --seed (same seed = same data). No real people.

- Insert 10 doctors as ms_admin (doctors cannot add doctors).

- Create patients with medisentinel.db.add_patient logged in as ms_doctor
(db.connect_as("ms_doctor")):

  age 18-90, gender from medisentinel.constants.GENDERS, renal/hepatic from ORGAN_FUNCTION
(mostly "normal"),

  0-3 conditions from db.list_conditions, 0-2 allergies to random medicines.

- Create prescriptions with db.create_prescription as ms_doctor: 1-4 DIFFERENT medicines
(random.sample),

  dose like "500 mg", frequency from FREQUENCIES. This makes the safety trigger run.

- Print totals: doctors, patients, prescriptions, and how many alerts the trigger raised.

B4 – ingestion/mongo_queries.py with top_reactions(n=10) and top_reported_drugs(n=10) using
aggregation

pipelines ($unwind, $group, $sort, $limit) on adverse_reports; print both as tables when run.

Run both scripts and show me the output.
```

Step 3 – check it yourself

The generator should report 100 patients, 300 prescriptions and a number of alerts above zero. `python ingestion/mongo_queries.py` must print two top-10 tables. You will run this live in the demo.

Step 4 – submit

Paste this prompt into Claude Code:

```
Run "python -m pytest" and make sure everything passes. Then show me "git status" and commit
only my

task's files with a clear message (no Co-Authored-By line, no mention of Claude). Push the
branch

feature/synthetic-data to origin and give me the GitHub link to open the pull request.
```

Open the link → **Create pull request** → tell Rishi. Wait for him to merge before starting the next task (or start the next branch from main if he says so).

Step 5 – understand it for the viva

Paste this prompt into Claude Code:

```
Explain generate_synthetic.py and mongo_queries.py for the viva: why we use synthetic data (privacy), why

prescriptions are created through db.create_prescription (the trigger runs), and what each aggregation stage

($unwind, $group, $sort, $limit) does with an example document. Then ask me 3 viva questions.
```

Fixing things Rishi asks for in review

Paste this prompt into Claude Code:

```
Rishi left review comments on my pull request. Here they are:

<paste the comments here>

Fix each one in my branch, run python -m pytest, commit (no Co-Authored-By line) and push to the same branch.
```

If something goes wrong

Problem	What to do
password authentication failed	Your .env passwords don't match the ones used in roles.sql – ask Claude to re-run sql/roles.sql with your .env passwords.
MongoDB connection refused	Start the service: Windows search → Services → MongoDB → Start.
ModuleNotFoundError: medisentinel	Activate .venv and run `pip install -e .`
Merge conflict / push rejected	Paste the error into Claude: "Explain this git error and fix it safely without losing my work." If unsure, ask Rishi.

Your checklist

	Done when
☐	Setup: check_setup says All good, tests pass
☐	B1: ~1000 labels and ~5000 reports in MongoDB, indexes created, PR merged

☐	B2: Medication table has several hundred rows, no duplicates, tests pass, PR merged
☐	B3/B4: 100 patients, 300 prescriptions, alerts raised; two top-10 tables printed; PR merged
☐	You can explain every script in your own words